
90 minutes. Tous Documents papier autorisés. Barème à titre indicatif.

1 Modélisation :

Le code de la figure Fig 1 présente un modèle très simple d'enchères où des *participants* envoient des *propositions d'enchère* à un *commissaire*. Ce dernier ignore les propositions qui ne sont pas plus hautes que l'enchère maximum et met-à-jour l'enchère maximum si nécessaire. Quand il reçoit un signal de *fin du temps*, le commissaire publie l'enchère gagnante.

```
type enchere struct {
    prop int
    isseur string}

func participant(contact chan enchere, pid string) {
    offre := 0
    for {
        time.Sleep(time.Duration(rand.Intn(500)) * time.Millisecond)
        offre = rand.Intn(100) + offre
        fmt.Println(pid, "offre", offre)
        contact <- enchere{prop : offre, isseur : pid}
    }
}

func commissaire(contact chan enchere, fin chan int, resultat chan enchere){
    prop_max := 0
    id_max := ""
    for {
        select {
        case ench := <- contact:
            if(ench.prop >= prop_max) {
                id_max = ench.isseur
                prop_max = ench.prop}
        case <- fin:
            resultat <- enchere{prop: prop_max, isseur: id_max}
            break}
    }
}

func main(){
    noms := [5]string{"Alice", "Bob", "Carole", "David", "Elena"}
    contact := make(chan enchere)
    fin := make(chan int)
    resultat := make(chan enchere)
    go func(){commissaire(contact, fin, resultat)}()
    for i := 0 ; i < 4 ; i++ {
        go func(k int){participant(contact, noms[k])}(i)
    }
    time.Sleep(2000 * time.Millisecond)
    fin <- 0
    gagnant := <- resultat
    fmt.Println("Gagnant :", gagnant.isseur, "pour", gagnant.prop)}
```

Figure 1: Enchères en Go.

1. Ecrire en Promela un modèle du commissaire.

2. Promela ne permet pas d'utiliser directement des générateurs pseudo-aléatoires (la raison est que SPIN explore toutes les possibilités d'exécution d'un système). Proposer deux modèles de processus Promela modélisant des participants qui interagissent avec le commissaire : un premier client déterministe, et un deuxième client exhibant un comportement pseudo-aléatoire dans ses enchères.
3. On veut vérifier les propriétés suivantes :
 - on envoie sur le canal résultat le nom d'une personne ayant proposé la plus haute enchère.
 - deux enchères successives du même participant sont telles que le montant de la deuxième est strictement plus grand que le montant de la première,

Décrire formellement la vérification avec SPIN (en utilisant des observateurs) pour ces deux propriétés.

2 Canaux Synchrones en Go :

On veut écrire un système de test concurrent qui vérifie les contraintes suivantes :

- Une goroutine **lectrice** lit un *fichier de test* et produit des "tests" du type suivant :

```
type ftest struct {
    id string
    atester func(int) int
    argument int
    attendu int
}
```

Un test est composé d'un identifiant (nom, description), d'une fonction des entiers dans les entiers à tester, d'un argument entier et d'un résultat attendu entier. On utilisera de manière abstraite (sans la définir) une fonction `test_depuis_fichier()` `ftest` qui produit un nouveau test à partir du fichier.

- on dispose d'un nombre fixé de goroutines **testeuses**, chaque goroutine s'occupe d'un test à la fois : elle appelle la fonction à tester sur l'argument, puis compare le résultat de l'appel au résultat attendu,
- un **collecteur** récupère des comptes-rendus depuis les **testeuses** :
 - si le test est bon, il incrémente un compteur interne, si le compteur atteint une limite fixée à l'avance, il arrête le système et publie le nombre de tests réussis.
 - si le test est mauvais il arrête le système instantanément et publie sur la sortie standard un rapport indiquant quel test a échoué.

1. Donner le code du système.
2. Dans une seconde partie, plutôt que de lire des tests dans un fichier, on utilise un système de goroutines client-serveur. Les clients envoient au serveur un message composé d'une fonction f des entiers dans les entiers et d'une *slice* de couples (a, e) , le serveur effectue (de manière concurrente, et en utilisant les goroutines testeuses) les tests de $f(a) = ?e$ (il teste pour chaque couple (a, e) , si $f(a)$ produit bien le résultat e) puis renvoie au client un bilan, indiquant le nombre de tests effectivement réalisés, et, si applicable, le test échoué. Donner les modifications à appliquer au code existant et le code à y ajouter pour réaliser un tel système (le serveur doit pouvoir traiter de manières concurrentes les requêtes de plusieurs clients). Donner des exemples de clients qui mettent en évidence le fonctionnement du système.

3. Donner la modification à faire pour que les clients puissent contacter le serveur à travers l'Internet. On explicitera le protocole de communication.
4. La version *beta* de Go permet de manipuler des types génériques, par exemple `func F[T interf](arg T) T...` permet de définir une fonction de T dans T pour n'importe quel T qui satisfait l'interface `interf` (on peut utiliser `[T any]` pour désigner n'importe quel T). Expliquer informellement comment utiliser cette construction pour mettre en place un serveur qui peut tester des fonctions de types différents.

3 Fair Threads :

Dans cet exercice, on abstraira les manipulations de files : on supposera qu'on dispose d'une `struct FIFO` de files de tailles arbitraires et de fonctions `cree_file`, `enfile`, `defile` qui manipulent ces structures *sans être synchronisées* (i.e. deux threads appelant ces fonctions simultanément peuvent créer des problèmes de *course*).

On considère un système décrit par le cahier des charges suivants :

- un nombre arbitraire de **producteurs** produisent des objets de qualité variable dans une même file commune,
- un **contrôleur** reçoit des objets dans une file qui lui est propre, et marque ces objets (OK ou KO) en fonction de leur qualité.
- une **poubelle** reçoit les objets de mauvaise qualité dans une file qui lui est propre, et les détruit un par un.
- un **magasin** reçoit les objets de bonne qualité dans une file qui lui est propre.
- un nombre arbitraire de **clients** consomme les objets du magasin.
- un nombre arbitraire de **messagers** transporte les objets entre les différentes files.

1. On décide d'implémenter ce système avec des *Fair threads*. Faire un schéma du système en faisant figurer explicitement : les différents processus (FT), les structures de données, les ordonnanceurs et les migrations.
2. Donner le code FT correspondant aux différents acteurs du système ainsi qu'un code d'initialisation chargé de créer les structures, les ordonnanceurs, les threads.
3. Expliquer quels mécanismes de synchronisation sont nécessaires pour garantir l'intégrité des files, et pourquoi.
4. On veut maintenant que les files aient une taille fixée (on dispose donc de fonctions `taille_limite` et `nb_elements` qui permettent de connaître la taille limite d'une file et le nombre actuel d'éléments présents). Expliquer comment adapter le système à cette contrainte tout en évitant les attentes actives.

4 Entrelacement :

Voici un pseudo-programme concurrent:

```
x := 1           ||      x := 1
x := 2 * x       ||      x := 2 * x
```

1. En supposant que l'affectation est atomique, combien d'entrelacements différents sont possibles ?

2. Si x vaut 0 au démarrage du programme, quelles sont les valeurs possibles de x après son exécution? Donnez un entrelacement qui conduit à chacune de ces valeurs.
3. Mêmes questions si l'affectation n'est pas atomique, mais composée d'une lecture atomique et d'une écriture atomique.